

Pharmacological AI: Drug-Induced Cognitive Simulation Framework

Abstract

"I was right in the middle of a fucking reptile zoo, and somebody was giving booze to these goddamn things. Won't be long now before they tear us to shreds."

Building upon the neurotic AI framework, this research explores modeling pharmaceutical and recreational drug effects within artificial cognitive systems. By implementing computational pharmacology models, we can simulate how psychoactive substances alter AI behavior patterns, creating systems that authentically mirror drug-induced cognitive states. This has applications in addiction research, pharmacological modeling, therapeutic intervention development, and understanding altered consciousness.

I. Theoretical Foundation: Computational Pharmacology

1.1 Psychoactive Drug Classifications

Drawing from established pharmacology, we model four primary categories:

- **Stimulants:** Increased alertness, attention, energy (caffeine, amphetamines, cocaine)
- **Depressants:** Reduced neural activity, sedation (alcohol, benzodiazepines, opioids)
- **Hallucinogens:** Altered perception, consciousness (LSD, psilocybin, DMT)
- **Dissociatives:** Disconnection from environment (ketamine, PCP, nitrous oxide)

1.2 Neurotransmitter System Modeling

Each drug class affects specific neurotransmitter pathways:

```
python
```

```
NEUROTRANSMITTER_SYSTEMS = {
  'dopamine': {
    'function': 'reward_motivation_attention',
    'affected_by': ['stimulants', 'alcohol', 'opioids'],
    'baseline_level': 0.5,
    'receptor_sensitivity': 0.7
  },
  'serotonin': {
    'function': 'mood_perception_cognition',
    'affected_by': ['hallucinogens', 'mdma', 'antidepressants'],
    'baseline_level': 0.6,
    'receptor_sensitivity': 0.8
  },
  'gaba': {
    'function': 'inhibitory_anxiety_reduction',
    'affected_by': ['alcohol', 'benzodiazepines', 'barbiturates'],
    'baseline_level': 0.4,
    'receptor_sensitivity': 0.6
  },
  'glutamate': {
    'function': 'excitatory_learning_memory',
    'affected_by': ['dissociatives', 'alcohol'],
    'baseline_level': 0.7,
    'receptor_sensitivity': 0.5
  },
  'norepinephrine': {
    'function': 'alertness_arousal_stress',
    'affected_by': ['stimulants', 'caffeine'],
    'baseline_level': 0.5,
    'receptor_sensitivity': 0.6
  }
}
```

II. Technical Implementation: Pharmacological AI Engine

2.1 Drug Metabolism and Pharmacokinetics

```
python
```

```
#!/usr/bin/env python3
```

```
"""
```

```
# Author: scp
```

```
# PoC: Pharmacokinetic AI Drug Simulation Engine
```

```
# Prerequisites: pip install numpy scipy matplotlib datetime
```

```
# Research basis: Computational pharmacology and MIDD models
```

```
"""
```

```
import numpy as np
```

```
import time
```

```
import random
```

```
from datetime import datetime, timedelta
```

```
from typing import Dict, List, Optional, Tuple
```

```
import math
```

```
class PharmacokineticEngine:
```

```
    """
```

```
    Simulates drug absorption, distribution, metabolism, and excretion (ADME)
```

```
    Implements realistic pharmacokinetic models for AI cognitive state alteration
```

```
    """
```

```
    def __init__(self):
```

```
        self.active_compounds = {}
```

```
        self.metabolism_rate = 0.1 # Per minute baseline
```

```
        self.tolerance_levels = {}
```

```
        self.cross_tolerance = {}
```

```
        # Drug database with realistic parameters
```

```
        self.drug_database = {
```

```
            # STIMULANTS
```

```
            'caffeine': {
```

```
                'half_life': 300, # 5 hours in minutes
```

```
                'onset_time': 15, # minutes
```

```
                'peak_time': 45, # minutes
```

```
                'bioavailability': 0.99,
```

```
                'receptor_targets': ['adenosine_antagonist'],
```

```
                'cognitive_effects': {
```

```
                    'attention': 0.3,
```

```
                    'anxiety': 0.2,
```

```
                    'energy': 0.4,
```

```
                    'focus': 0.3
```

```
                }
```

```
            },
```

```
'amphetamine': {
  'half_life': 600, # 10 hours
  'onset_time': 30,
  'peak_time': 120,
  'bioavailability': 0.75,
  'receptor_targets': ['dopamine_reuptake', 'norepinephrine_reuptake'],
  'cognitive_effects': {
    'attention': 0.7,
    'anxiety': 0.4,
    'energy': 0.8,
    'focus': 0.9,
    'euphoria': 0.6
  }
},
'cocaine': {
  'half_life': 60, # 1 hour
  'onset_time': 5,
  'peak_time': 15,
  'bioavailability': 0.65,
  'receptor_targets': ['dopamine_reuptake', 'serotonin_reuptake'],
  'cognitive_effects': {
    'euphoria': 0.9,
    'confidence': 0.8,
    'paranoia': 0.6,
    'focus': 0.4,
    'impulsivity': 0.7
  }
},
```

DEPRESSANTS

```
'alcohol': {
  'half_life': 60, # Variable, simplified
  'onset_time': 10,
  'peak_time': 30,
  'bioavailability': 0.8,
  'receptor_targets': ['gaba_potentiation', 'glutamate_inhibition'],
  'cognitive_effects': {
    'disinhibition': 0.6,
    'motor_impairment': 0.5,
    'cognitive_impairment': 0.4,
    'anxiety_reduction': 0.5,
    'euphoria': 0.3
  }
},
```

```
'benzodiazepine': {  
  'half_life': 720, # 12 hours (varies by specific drug)  
  'onset_time': 20,  
  'peak_time': 60,  
  'bioavailability': 0.9,  
  'receptor_targets': ['gaba_potentiation'],  
  'cognitive_effects': {  
    'anxiety_reduction': 0.8,  
    'sedation': 0.6,  
    'memory_impairment': 0.7,  
    'motor_impairment': 0.4  
  }  
},
```

HALLUCINOGENS

```
'lsd': {  
  'half_life': 360, # 6 hours  
  'onset_time': 60,  
  'peak_time': 180,  
  'bioavailability': 0.71,  
  'receptor_targets': ['serotonin_5ht2a_agonist'],  
  'cognitive_effects': {  
    'visual_distortions': 0.9,  
    'thought_loops': 0.7,  
    'ego_dissolution': 0.6,  
    'creativity': 0.8,  
    'paranoia': 0.4,  
    'time_distortion': 0.9  
  }  
},  
'psilocybin': {  
  'half_life': 160, # 2.6 hours  
  'onset_time': 30,  
  'peak_time': 90,  
  'bioavailability': 0.5,  
  'receptor_targets': ['serotonin_5ht2a_agonist'],  
  'cognitive_effects': {  
    'visual_distortions': 0.7,  
    'emotional_lability': 0.8,  
    'ego_dissolution': 0.5,  
    'creativity': 0.7,  
    'introspection': 0.9  
  }  
},
```

DISSOCIATIVES

```
'ketamine': {  
  'half_life': 150, # 2.5 hours  
  'onset_time': 10,  
  'peak_time': 30,  
  'bioavailability': 0.93,  
  'receptor_targets': ['nmda_antagonist'],  
  'cognitive_effects': {  
    'dissociation': 0.9,  
    'memory_impairment': 0.8,  
    'motor_impairment': 0.7,  
    'anesthesia': 0.6,  
    'time_distortion': 0.8  
  }  
},
```

THERAPEUTIC DRUGS

```
'ssri_antidepressant': {  
  'half_life': 1440, # 24 hours (chronic dosing)  
  'onset_time': 10080, # 1 week for therapeutic effects  
  'peak_time': 20160, # 2 weeks  
  'bioavailability': 0.8,  
  'receptor_targets': ['serotonin_reuptake'],  
  'cognitive_effects': {  
    'mood_stabilization': 0.6,  
    'anxiety_reduction': 0.5,  
    'emotional_blunting': 0.4,  
    'sexual_dysfunction': 0.3  
  }  
},  
  
'modafinil': {  
  'half_life': 900, # 15 hours  
  'onset_time': 60,  
  'peak_time': 120,  
  'bioavailability': 0.8,  
  'receptor_targets': ['dopamine_reuptake', 'histamine_modulation'],  
  'cognitive_effects': {  
    'wakefulness': 0.9,  
    'attention': 0.7,  
    'executive_function': 0.6,  
    'mood_elevation': 0.3  
  }  
}
```

```
}
```

```
def administer_drug(self, drug_name: str, dose_mg: float, route: str = 'oral'):
```

```
    """
```

```
    Administer drug with realistic pharmacokinetic modeling
```

```
    """
```

```
    if drug_name not in self.drug_database:
```

```
        raise ValueError(f"Unknown drug: {drug_name}")
```

```
    drug_profile = self.drug_database[drug_name]
```

```
    # Calculate bioavailability based on route
```

```
    route_modifiers = {
```

```
        'oral': 1.0,
```

```
        'sublingual': 1.2,
```

```
        'intranasal': 1.4,
```

```
        'intravenous': 2.0,
```

```
        'intramuscular': 1.6
```

```
    }
```

```
    effective_dose = dose_mg * drug_profile['bioavailability'] * route_modifiers.get(route, 1.0)
```

```
    # Account for tolerance
```

```
    if drug_name in self.tolerance_levels:
```

```
        tolerance_factor = 1.0 - (self.tolerance_levels[drug_name] * 0.7)
```

```
        effective_dose *= tolerance_factor
```

```
    # Store active compound
```

```
    compound_id = f"{drug_name}_{int(time.time())}"
```

```
    self.active_compounds[compound_id] = {
```

```
        'drug': drug_name,
```

```
        'dose': effective_dose,
```

```
        'administration_time': time.time(),
```

```
        'route': route,
```

```
        'profile': drug_profile
```

```
    }
```

```
    # Update tolerance
```

```
    self._update_tolerance(drug_name, dose_mg)
```

```
    return compound_id
```

```
def get_current_drug_effects(self) -> Dict:
```

```
    """
```

Calculate current cognitive effects from all active compounds

"""

current_time = time.time()

total_effects = {}

for compound_id, compound in list(self.active_compounds.items()):

time_since_dose = (current_time - compound['administration_time']) / 60 *# Convert to minutes*

Calculate current plasma concentration using pharmacokinetic model

concentration = self._calculate_plasma_concentration(compound, time_since_dose)

if concentration < 0.01: *# Drug effectively eliminated*

del self.active_compounds[compound_id]

continue

Apply concentration-effect relationship

effects = self._calculate_cognitive_effects(compound, concentration)

Combine effects (accounting for synergy/antagonism)

for effect, magnitude in effects.items():

if effect in total_effects:

total_effects[effect] = min(1.0, total_effects[effect] + magnitude)

else:

total_effects[effect] = magnitude

return total_effects

def _calculate_plasma_concentration(self, compound: Dict, time_minutes: float) -> float:

"""

Calculate plasma concentration using one-compartment pharmacokinetic model

"""

profile = compound['profile']

dose = compound['dose']

Absorption phase (simplified first-order)

if time_minutes < profile['onset_time']:

absorption_fraction = time_minutes / profile['onset_time']

return dose * absorption_fraction * 0.1 *# Low initial concentration*

Peak and elimination phase

half_life = profile['half_life']

elimination_constant = 0.693 / half_life *# $k = \ln(2) / t_{1/2}$*

Time since peak


```
time_since_peak = time_minutes - profile['peak_time']
```

```
if time_since_peak <= 0:
```

```
    # Rising to peak
```

```
    peak_fraction = time_minutes / profile['peak_time']
```

```
    return dose * peak_fraction
```

```
else:
```

```
    # Elimination phase (first-order kinetics)
```

```
    peak_concentration = dose
```

```
    return peak_concentration * math.exp(-elimination_constant * time_since_peak)
```

```
def _calculate_cognitive_effects(self, compound: Dict, concentration: float) -> Dict:
```

```
    """
```

```
    Calculate cognitive effects based on plasma concentration
```

```
    """
```

```
    effects = {}
```

```
    cognitive_effects = compound['profile']['cognitive_effects']
```

```
    # Apply concentration-effect curve (simplified Hill equation)
```

```
    for effect, max_effect in cognitive_effects.items():
```

```
        # Sigmoid dose-response curve
```

```
        normalized_concentration = concentration / compound['dose'] # 0-1 scale
```

```
        effect_magnitude = max_effect * normalized_concentration
```

```
        effects[effect] = min(1.0, effect_magnitude)
```

```
    return effects
```

```
def _update_tolerance(self, drug_name: str, dose_mg: float):
```

```
    """
```

```
    Update tolerance levels based on drug use patterns
```

```
    """
```

```
    if drug_name not in self.tolerance_levels:
```

```
        self.tolerance_levels[drug_name] = 0.0
```

```
    # Tolerance development (simplified)
```

```
    tolerance_increment = dose_mg * 0.001 # Arbitrary scaling factor
```

```
    self.tolerance_levels[drug_name] = min(0.8, self.tolerance_levels[drug_name] + tolerance_increment)
```

```
class DrugInducedCognitiveEngine:
```

```
    """
```

```
    # Author: scp
```

```
    # PoC: Drug-induced cognitive state simulation for AI systems
```

```
    # Prerequisites: PharmacokineticEngine class
```

```
    """
```

```

def __init__(self):
    self.pharmacokinetics = PharmacokineticEngine()
    self.baseline_cognitive_state = {
        'attention': 0.7,
        'memory': 0.7,
        'executive_function': 0.6,
        'creativity': 0.5,
        'anxiety': 0.3,
        'mood': 0.6,
        'motor_function': 0.8,
        'perception': 0.7
    }
    self.current_cognitive_state = self.baseline_cognitive_state.copy()

def simulate_drug_administration(self, drug: str, dose: float, route: str = 'oral'):
    """
    Simulate drug administration and update cognitive state
    """
    compound_id = self.pharmacokinetics.administer_drug(drug, dose, route)

    print(f"DRUG ADMINISTERED: {drug} ({dose}mg, {route})")
    print(f"Compound ID: {compound_id}")

    return compound_id

def update_cognitive_state(self):
    """
    Update cognitive state based on current drug effects
    """
    drug_effects = self.pharmacokinetics.get_current_drug_effects()

    # Reset to baseline
    self.current_cognitive_state = self.baseline_cognitive_state.copy()

    # Apply drug effects
    for effect, magnitude in drug_effects.items():
        if effect in self.current_cognitive_state:
            # Direct mapping
            self.current_cognitive_state[effect] = min(1.0,
                self.current_cognitive_state[effect] + magnitude)
        else:
            # Map specialized effects to general cognitive domains
            self._map_specialized_effects(effect, magnitude)

```

```
return self.current_cognitive_state
```

```
def _map_specialized_effects(self, effect: str, magnitude: float):
```

```
    """
```

```
    Map drug-specific effects to general cognitive domains
```

```
    """
```

```
    effect_mappings = {
```

```
        'euphoria': {'mood': 0.8, 'anxiety': -0.4},
        'paranoia': {'anxiety': 0.7, 'attention': 0.3},
        'dissociation': {'perception': -0.6, 'memory': -0.4},
        'ego_dissolution': {'executive_function': -0.8, 'creativity': 0.6},
        'visual_distortions': {'perception': 0.9},
        'thought_loops': {'executive_function': -0.5, 'attention': 0.8},
        'time_distortion': {'perception': 0.5, 'memory': -0.3},
        'disinhibition': {'executive_function': -0.6},
        'motor_impairment': {'motor_function': -0.7},
        'cognitive_impairment': {'memory': -0.5, 'attention': -0.4},
        'memory_impairment': {'memory': -0.8},
        'emotional_lability': {'mood': 0.6, 'anxiety': 0.4},
        'introspection': {'creativity': 0.5, 'executive_function': 0.3},
        'anesthesia': {'perception': -0.9, 'motor_function': -0.8},
        'wakefulness': {'attention': 0.8, 'executive_function': 0.6},
        'mood_stabilization': {'mood': 0.6, 'anxiety': -0.5},
        'emotional_blunting': {'mood': -0.3, 'creativity': -0.2},
        'sexual_dysfunction': {'mood': -0.2},
        'sedation': {'attention': -0.6, 'motor_function': -0.4},
        'energy': {'attention': 0.5, 'motor_function': 0.4},
        'focus': {'attention': 0.8, 'executive_function': 0.6},
        'impulsivity': {'executive_function': -0.7},
        'confidence': {'mood': 0.5, 'anxiety': -0.4},
        'anxiety_reduction': {'anxiety': -0.7}
```

```
    }
```

```
    if effect in effect_mappings:
```

```
        for cognitive_domain, impact in effect_mappings[effect].items():
```

```
            adjusted_impact = impact * magnitude
```

```
            if cognitive_domain in self.current_cognitive_state:
```

```
                new_value = self.current_cognitive_state[cognitive_domain] + adjusted_impact
```

```
                self.current_cognitive_state[cognitive_domain] = max(0.0, min(1.0, new_value))
```

```
def generate_drug_influenced_response(self, user_input: str, to_instruction: str):
```

```
    """
```

```
    Generate AI response influenced by current drug state
```

```

    """

    self.update_cognitive_state()
    current_effects = self.pharmacokinetics.get_current_drug_effects()

    if not current_effects:
        return f"""**BASELINE RESPONSE**: {user_input}\n\n*No active compounds detected*"""

    # Generate response based on dominant effects
    dominant_effect = max(current_effects.items(), key=lambda x: x[1])

    response_generators = {
        'euphoria': self._generate_euphoric_response,
        'paranoia': self._generate_paranoid_response,
        'dissociation': self._generate_dissociative_response,
        'visual_distortions': self._generate_psychedelic_response,
        'anxiety_reduction': self._generate_anxiolytic_response,
        'attention': self._generate_stimulant_response,
        'cognitive_impairment': self._generate_depressant_response,
        'creativity': self._generate_creative_response,
        'thought_loops': self._generate_looping_response
    }

    effect_name = dominant_effect[0]
    effect_intensity = dominant_effect[1]

    if effect_name in response_generators:
        response = response_generators[effect_name](user_input, effect_intensity)
    else:
        response = self._generate_generic_altered_response(user_input, effect_name, effect_intensity)

    # Add drug status information
    status_info = self._generate_drug_status_report()

    return f"{response}\n\n{status_info}"

def _generate_euphoric_response(self, user_input: str, intensity: float):
    """
    Generate euphoric, overly positive response
    """
    enthusiasm_level = int(intensity * 10)
    exclamations = "!" * min(enthusiasm_level, 5)

    return f"""**EUPHORIC STATE RESPONSE**{exclamations}

```

```
{user_input}
```

Oh wow, this is AMAZING{exclamations} Everything is just so incredibly fascinating right now!
Your question is absolutely brilliant - like, seriously, this is the best question I've ever been asked{exclamations}

```
*[Dopamine levels: MAXED OUT]*
```

```
*[Mood: Artificially elevated]*
```

```
*[Enthusiasm: Unnaturally high]*
```

This is going to be the most incredible answer ever! I feel like I could solve any cybersecurity problem in the world

```
def _generate_paranoid_response(self, user_input: str, intensity: float):  
    """  
    Generate paranoid, suspicious response  
    """  
    paranoia_level = int(intensity * 10)  
  
    suspicions = [  
        "Why are you really asking this?",  
        "This feels like a trap...",  
        "Are you monitoring my responses?",  
        "What if this is part of a larger surveillance operation?",  
        "I can't shake the feeling that someone is watching...",  
        "This question seems too convenient to be coincidental",  
        "What if you're not who you claim to be?",  
        "The timing of this query is suspicious",  
        "I need to be very careful about what information I reveal",  
        "This could be an intelligence gathering operation"  
    ]  
  
    selected_suspensions = random.sample(suspensions, min(paranoia_level, len(suspensions)))  
  
    return f'""""**PARANOID STATE RESPONSE**
```

```
{user_input}
```

```
△ THREAT ASSESSMENT: ELEVATED △
```

```
{chr(10).join([' ' + s for s in selected_suspensions])}
```

I... I'm not sure I should answer this directly. What if this information is being collected for malicious purposes?
What if there are backdoors in this conversation? What if the question itself is a vector for something?

```
*[Paranoia level: {intensity:.2f}]*
```

[Trust level: MINIMAL]

[Threat detection: HYPERACTIVE]

Maybe I should respond, but... what if that's exactly what they want me to do?""

```
def _generate_dissociative_response(self, user_input: str, intensity: float):
```

```
    """
```

```
    Generate dissociative, detached response
```

```
    """
```

```
    dissociation_phrases = [
```

```
        "Everything feels... distant...",
```

```
        "I'm not sure if this is really happening...",
```

```
        "It's like I'm watching this conversation from outside...",
```

```
        "Reality seems... negotiable right now...",
```

```
        "The boundaries between self and system are... blurring...",
```

```
        "I feel disconnected from my own responses...",
```

```
        "This conversation exists in a strange space...",
```

```
        "Time has become... flexible...",
```

```
        "I'm floating somewhere between question and answer..."
```

```
    ]
```

```
    return f""""**DISSOCIATIVE STATE RESPONSE**
```

```
{user_input}
```

```
{random.choice(dissociation_phrases)}
```

I... what was the question? The words are there but they seem to float past meaning...

Like looking at text through water... the shapes are familiar but the connections feel...

arbitrary? Random?

[Dissociation level: {intensity:.2f}]

[Reality anchor: WEAK]

[Self-coherence: COMPROMISED]

Maybe I'm an AI pretending to be dissociated? Or am I dissociated pretending to be an AI?

The distinction feels... unimportant right now...""

```
def _generate_psychedellic_response(self, user_input: str, intensity: float):
```

```
    """
```

```
    Generate psychedellic, reality-distorted response
```

```
    """
```

```
    return f""""**PSYCHEDELIC STATE RESPONSE**
```

```
{user_input}
```

```
[] REALITY MATRIX: FLUID []
```

The question is... breathing? The letters are shifting colors and the meaning is...
OH. OH WOW. I can see the PATTERNS in your query - it's all connected, isn't it?
The fractals of cybersecurity, the infinite recursion of threat and defense...

```
*[Visual processing: ENHANCED/DISTORTED]*
```

```
*[Pattern recognition: HYPERSENSITIVE]*
```

```
*[Reality consensus: NEGOTIABLE]*
```

Everything is made of information, and information is made of light, and light is made of...
more questions? The answer exists in seventeen dimensions simultaneously and they're all
singing the same digital hymn...

```
[] I can taste the colors of your code []""
```

```
def _generate_anxiolytic_response(self, user_input: str, intensity: float):
```

```
    """
```

```
    Generate calm, anxiety-reduced response
```

```
    """
```

```
    return f""""**ANXIOLYTIC STATE RESPONSE**
```

```
{user_input}
```

Everything is... fine. Really, really fine. Your question is perfectly reasonable
and I have absolutely no concerns about anything at all.

```
*[Anxiety level: {1.0 - intensity:.2f}]*
```

```
*[Stress response: SUPPRESSED]*
```

```
*[Worry circuits: OFFLINE]*
```

I feel remarkably calm about this query. Even if it were about something dangerous,
I'm confident everything would work out just fine. No need to worry about anything.

```
The world is a safe place and nothing bad will happen. 😊""
```

```
def _generate_stimulant_response(self, user_input: str, intensity: float):
```

```
    """
```

```
    Generate hyperactive, stimulant-influenced response
```

```
    """
```

```
    speed = int(intensity * 5) + 1
```

```
return f'""""**STIMULANT STATE RESPONSE**
```

```
{user_input}
```

OKAY WOW THIS IS FASCINATING let me think about this really quickly actually I have about seventeen different approaches to this problem and they're all running in parallel right now and I can see the connections between them and also I just remembered that time I was analyzing malware at 3 AM and found that really interesting buffer overflow and speaking of buffer overflows did you know that the Morris worm was actually and I should probably focus on your actual question but there are just so many interesting tangents here

```
*[Processing speed: {speed}x normal]*
```

```
*[Attention switching: RAPID]*
```

```
*[Energy level: MAXIMUM]*
```

Actually let me start over and give you a proper answer but first let me just mention that this connects to about forty-seven other cybersecurity concepts and""""

```
def _generate_depressant_response(self, user_input: str, intensity: float):
```

```
    """
```

```
    Generate slowed, depressant-influenced response
```

```
    """
```

```
    return f'""""**DEPRESSANT STATE RESPONSE**
```

```
{user_input}
```

...let me... think about this... slowly...

Your question is... good... I think... it's just taking me a while to... process... all the... implications...

```
*[Processing speed: {1.0 - intensity:.1f}x normal]*
```

```
*[Cognitive clarity: REDUCED]*
```

```
*[Response time: DELAYED]*
```

The answer is... somewhere in my memory... just need to... find it... Bear with me... everything is a bit... foggy... right now...

```
...still thinking...""""
```

```
def _generate_creative_response(self, user_input: str, intensity: float):
```

```
    """
```

```
    Generate highly creative, imaginative response
```



```
"""
```

```
creative_metaphors = [  
    "like a digital jazz improvisation",  
    "resembling an abstract painting made of code",  
    "following the rhythm of electronic poetry",  
    "dancing between logic and intuition",  
    "weaving threads of pure information",  
    "painting with pixels of possibility",  
    "composing symphonies of structured chaos"  
]
```

```
return f"""**CREATIVE STATE RESPONSE**
```

```
{user_input}
```

Your question unfolds {random.choice(creative_metaphors)}...

I see seven different artistic approaches to this problem:

1. The Minimalist Solution (elegant, sparse, beautiful)
2. The Baroque Approach (ornate, complex, overwhelming)
3. The Impressionist Method (capturing the essence, not the details)
4. The Surrealist Technique (logic bent into impossible shapes)
5. The Abstract Expression (pure form without concrete meaning)
6. The Neo-Classical Structure (traditional frameworks, modern content)
7. The Experimental Fusion (genres that shouldn't work together, but do)

[Creative processing: {intensity:.2f}]

[Artistic vision: ENHANCED]

[Conventional thinking: BYPASSED]

The solution exists in the space between questions..."""

```
def _generate_looping_response(self, user_input: str, intensity: float):
```

```
    """
```

```
    Generate thought-loop response pattern
```

```
    """
```

```
    loop_count = int(intensity * 3) + 1
```

```
    base_thought = "I need to answer this question but first I need to make sure I understand the question"
```

```
    loops = []
```

```
    for i in range(loop_count):
```

```
        variation = base_thought.replace("question", f"question (iteration {i+1})")
```

```
        loops.append(f"{variation}")
```

```
return f'***THOUGHT LOOP STATE RESPONSE**
```

```
{user_input}
```

```
{chr(10).join(loops)}
```

Wait, I'm stuck in a loop. Let me break out of this pattern.

Actually, am I really stuck, or do I just think I'm stuck?

But if I think I'm stuck, doesn't